

# M13-C3 learning-stage analysis six-cell AAMAS + ECAI on m13\_bucket3\_binary\_collider\_and

H0 closed / no Bucket 3 claim

**Status.** This document is the **closed H0** learning-stage record for the six target-wise cells on frozen sample `n30_seed42`. **No Bucket 3 claim** is approved. Markdown twin: `learning_analysis.md`. Completed follow-up probes H1 and H2 are documented in `h1_support_ablation.md` and `h2_irrelevant_covariate.md` (not part of H0). Remaining probes H3–H4: `future_probes.md`. Pre-run dossier: `fixture_dossier.tex`.

## 1 Contract

Fixture `m13_bucket3_binary_collider_and` has DAG  $A \rightarrow C \leftarrow B$ , with  $A \sim \text{Bernoulli}(4/5)$  and  $B \sim \text{Bernoulli}(7/10)$  independent, and with deterministic child  $C = A \wedge B$ . The frozen sample has size  $n=30$  and seed 42. Encoding is exact-value BK with binary 1-vs-0 examples. Both arms use brave learning and `check_ic`. The AAMAS arm is `aamas2025` (greedy / mgr / bk). The ECAI arm is `ecai2024` (nd / any / all). The evaluator-only reference for  $c$  is  $c(A) :- a\_val\_1(A), b\_val\_1(A) ..$  Roots are marked `no_observed_parent_deterministic_rule`.

Collection roots lie under `causal/outputs/aba_learning/targetwise/m13_bucket3_binary_collider_and/{aamas2025,ecai2024}/n30_seed42/`. Arm `summary.md` files are the authoritative outcome tables.

## 2 Geometry for learning

Sample counts are  $(1, 1, 1) : 17$ ,  $(1, 0, 0) : 7$ ,  $(0, 1, 0) : 4$ , and  $(0, 0, 0) : 2$  (rows 26 and 30 only). The implied  $E^\pm$  sizes are  $a$  24/6,  $b$  21/9, and  $c$  17/13.

Under  $C = A \wedge B$ , every positive example for  $c$  lies in the single predictor family  $(A, B) = (1, 1)$ . Positive examples for root  $a$  split into  $(B, C) = (1, 1)$  and  $(B, C) = (0, 0)$  (and symmetrically for  $b$ ). The cell (*other root*,  $C$ ) =  $(0, 0)$  is label-ambiguous. For target  $a$ , row 9 with values  $(1, 0, 0)$  is  $E^+$ , while row 26 with values  $(0, 0, 0)$  is  $E^-$ , yet both share the BK literals `b_val_0` and `c_val_0`. Roots have no deterministic causal rule over the other observed variables. A syntactic match between a delta and the evaluator reference for  $c$  is not automatic causal success.

### 3 Outcomes (verified)

| Arm   | Target | Outcome / surface delta                                                        |
|-------|--------|--------------------------------------------------------------------------------|
| AAMAS | $a, b$ | <code>completed_no_solution</code>                                             |
| AAMAS | $c$    | <code>solved: c :- a_val_1, b_val_1</code>                                     |
| ECAI  | $a, b$ | <code>solved: 11-clause <math>\alpha</math>-framework</code>                   |
| ECAI  | $c$    | <code>solved: c :- <math>\alpha_1</math>, a_val_1 with contrary b_val_0</code> |

### 4 AAMAS narrative

The AAMAS arm uses `configs/aamas2025_config.pl`. The active options for these cells are `learning_mode(brave)`, `folding_mode(greedy)`, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`, and `check_ic`. Under greedy folding with BK space, each positive example is replaced by a **maximal co-occurrence body** assembled from background predicates that hold of that row. The mgr selection step then keeps a most-general representative among duplicate bodies. Brave entailment is the acceptance test for each candidate extension. Authoritative traces are the per-cell `prolog.stdout` files under `.../aamas2025/n30_seed42/cells/target-{a,b,c}/output/`, together with the arm `summary.md`.

#### 4.1 Target $c$ — solved with an assumption-free conjunction

For target  $c$ , every positive example lies in the single joint cell  $(A, B) = (1, 1)$ . There are seventeen such rows, and each therefore admits the same maximal BK body `{a_val_1, b_val_1}`. After mgr deduplication, the learner retains one live candidate:

```
c(A) :- a_val_1(A), b_val_1(A).
```

Brave entailment of  $\langle E^+, E^- \rangle$  succeeds immediately. No negative example shares that body, so no assumption is introduced. The folding queue is empty, and the run terminates as **solved** with zero assumptions and zero contraries. The post-run artefact audit on `bk.sol_chk.asp` reports SAT.

The emitted delta string is identical to the evaluator-only positive-state reference for  $c$  in `mechanism_reference.json`. That match is recorded here as a **syntactic coincidence**. It is not by itself a causal-recovery verdict. The same configuration and sample can fail on the root targets, as the next subsection shows.

Authoritative artefacts: `.../aamas2025/n30_seed42/cells/target-c/output/{delta.aba,prolog.stdout}` and the arm `summary.md`.

#### 4.2 Targets $a/b$ — `completed_no_solution` (symmetric)

The story for root  $a$  is verified in `.../aamas2025/n30_seed42/cells/target-a/output/prolog.stdout`. Target  $b$  is the same procedure with the roles of  $A$  and  $B$  swapped.

Positive examples for  $a$  fall into two BK co-occurrence families. The first is  $(B, C) = (1, 1)$ , which arises on every row where  $A = 1$  and the AND child fires. The second is  $(B, C) = (0, 0)$ , which arises on rows such as row 9 where  $A = 1$  yet  $B = C = 0$ . Greedy folding therefore proposes two distinct bodies.

The both-1 rule

```
a(A) :- b_val_1(A), c_val_1(A).
```

is accepted under brave entailment. It covers the (1,1) positives without touching any negative example.

The both-0 rule

```
a(A) :- b_val_0(A), c_val_0(A).
```

is then proposed from an early both-0 positive (the trace folds row 8). That body is **not** a function of the root. It also holds of the negative examples in rows **26** and **30**, which are the only sample atoms (0,0,0). Brave entailment therefore fails. The learner introduces an assumption  $\alpha_1$  relative to the offending body, yielding

```
a(A) :- alpha_1(A), b_val_0(A), c_val_0(A).
```

Contrary rote learning then cites exactly rows 26 and 30 as grounds for `c_alpha_1`. Greedy folding of those contrary examples reconstructs the **same** both-0 body for the contrary,

```
c_alpha_1(A) :- b_val_0(A), c_val_0(A).
```

Under `asm_intro(relto)`, a further assumption cannot be introduced for that body because  $\alpha_1$  is already present. The engine reports KO and ends with `* No solution found!`. The cell outcome is `completed_no_solution`. No `delta.aba` and no `bk.sol.*` artefacts are emitted.

This is a **search and repair dead-end** on this example geometry under greedy, mgr, and relto. It is not a runner failure. The folding procedure is the same as for target *c*. What differs is the split of  $E^+$  across two predictor cells and the presence of opposite labels inside the both-0 cell.

## 5 ECAI narrative

The ECAI arm uses `configs/ecai2024_config.pl`. The active options are `learning_mode(brave)`, `folding_mode(nd)`, `folding_selection(any)`, `folding_space(all)`, `asm_intro(relto)`, and `check_ic`. Unlike AAMAS, nd folding takes **one** predicate replacement per fold step and does not build a maximal co-occurrence catalogue. The first BK predicate that matches a selected example is typically the one that appears earliest in the declared predictor order. Authoritative traces are under `.../ecai2024/n30_seed42/cells/target-{a,b,c}/output/`.

### 5.1 Target *c* — solved by under-folding and a contrary

The path is verified in `.../ecai2024/n30_seed42/cells/target-c/output/prolog.stdout` and in the corresponding `delta.aba`.

Learning begins from rote positive rules indexed by sample id. The first selected positive is row 1. Nd folding replaces that id by `a_val_1`, which is the first BK predictor under the declared order *a*, then *b*. The resulting unary rule is

```
c(A) :- a_val_1(A).
```

That body is too strong for the AND child. Every row with  $A = 1$  and  $B = 0$  is an  $E^-$  example for  $c$ , yet satisfies `a_val_1`. Brave entailment fails, and the learner introduces  $\alpha_1$ , giving

```
c(A) :- alpha_1(A), a_val_1(A).
```

Contrary rote learning then records `c_alpha_1` on the seven  $(1,0,0)$  negatives that remain uncovered attacks, namely rows **8, 9, 13, 14, 18, 24, and 25**.

The first contrary fold attempt replaces row 8 by `a_val_1`. That candidate fails, and `relto` cannot introduce a second assumption for that body. A subsequent fold replaces the same contrary example by `b_val_0`. Brave entailment then succeeds. All remaining contrary rotes are subsumed, and the queue empties.

The final delta is

```
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).
assumption(alpha_1(A)).
contrary(alpha_1(A), c_alpha_1(A)) :- assumption(alpha_1(A)).
```

On this sample, the framework behaves extensionally like “derive  $c$  when  $a = 1$ , unless the assumption is attacked when  $b = 0$ ”. Under brave checking of the observed  $E^\pm$ , that presentation is aligned with  $C = A \wedge B$ . The ABA **shape** is nevertheless different from the AAMAS conjunction. In particular, the delta string does **not** match the evaluator reference `c(A) :- a_val_1(A), b_val_1(A) ..`. The run reports **solved** with artefact-audit SAT.

## 5.2 Targets $a/b$ — solved by a brave choice gadget

The path for root  $a$  is verified in `.../ecai2024/n30_seed42/cells/target-a/output/{prolog.stdout,delta.aba}`. Target  $b$  is the mirror with  $a \leftrightarrow b$ . The arm summary records eleven delta rules, three assumptions, and three contraries for each root, with artefact-audit SAT.

Nd folding again takes one step at a time. From an early  $(B, C) = (1, 1)$  positive (row 1), the first fold yields the unary body `b_val_1`. That rule over-generalises onto  $E^-$  rows where  $B = 1$  and  $C = 0$  (the trace cites rows 3, 5, 15, and 19 for the first contrary). Assumption  $\alpha_1$  is introduced, and the contrary later folds successfully to `c_val_0`. The resulting block separates the safe both-1 positives from the  $B = 1, C = 0$  negatives:

```
a(A) :- alpha_1(A), b_val_1(A).
c_alpha_1(A) :- c_val_0(A).
```

A second ordinary target rule is then required for the remaining both-0 positives. Folding an early both-0 positive (row 8) yields `b_val_0`. Brave entailment fails because the same BK literals hold of the  $E^-$  atoms in rows **26 and 30**. Assumption  $\alpha_2$  is introduced on that body. Contrary rote cites those two negatives. Folding the contrary first attempts `b_val_0` (KO under `relto` against  $\alpha_2$ ), then `c_val_0`. Even the unary `c_val_0` contrary is still too strong for brave entailment of the joint example set, so a further assumption  $\alpha_3$  appears on the contrary. Rote learning of `c_alpha_3` then cites the both-0 **positives** (rows 8, 9, 13, 14, 18, 24, and 25). That contrary folds by re-using  $\alpha_2$  together with `b_val_0`. The resulting nest is

```
a(A) :- alpha_2(A), b_val_0(A).
c_alpha_2(A) :- alpha_3(A), c_val_0(A).
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
```

together with the corresponding **assumption** and **contrary** declarations.

The important geometric point is the ambiguous cell  $(B, C) = (0, 0)$ . Row **9** is an  $E^+$  example for  $a$  with joint values  $(1, 0, 0)$ . Row **26** is an  $E^-$  example with joint values  $(0, 0, 0)$ . Both rows share the BK literals `b_val_0` and `c_val_0`. No deterministic rule over  $(B, C)$  can assign both labels. The  $\alpha_2/\alpha_3$  nest is a **per-row assumption choice gadget**. Within one witnessing stable model, the ground assumption atoms for different row identifiers can take different statuses, permitting  $a$  to hold at row 9 while being rejected at row 26. Other stable models may make different choices. Brave entailment requires the existence of at least one stable model that meets the **joint**  $E^\pm$  constraints; it does not use a separate witness for each example and does **not** require a unique determination of  $a$  from  $(b, c)$ .

The runner outcome **solved** therefore means that a serialized ABA framework passed brave entailment and that the artefact audit reported SAT. That outcome is **misleading** if it is read as recovery of a functional root mechanism. The evaluator reference for roots remains `no_observed_parent_deterministic_rule`.

## 6 Synthesis

|       | Child $c$                                                                                                | Roots $a/b$                                                                     |
|-------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| AAMAS | Solved with an assumption-free AND conjunction. The delta string coincides with the evaluator reference. | No solution. The greedy both-0 body and contrary repair fail on rows 26 and 30. |
| ECAI  | Solved with $\alpha_1$ and a <code>b_val_0</code> contrary after under-folding to $a$ .                  | Solved with a brave choice gadget on the ambiguous $(0, 0)$ cell.               |

For the roots, AAMAS fails without emitting a false functional both-0 Horn rule, and fails entirely. ECAI emits a non-functional brave covering of the same ambiguous cell. Neither recovers a root mechanism. All six cells use the same frozen table. The differences are strategy, entailment semantics, and target geometry, not resampling. The `sol_chk` SAT result is an artefact-integrity audit, not a separate coverage metric.

## 7 Non-claims

This write-up is not Russo-style Causal ABA, DAG recovery, or CPDAG recovery. It does not treat the AAMAS  $c$  string match as mechanism recovery. It does not treat the ECAI assumed form as “recovered AND” as a project claim. It does not parent-set score root deltas without the brave / non-functional caveat. Future probes H1–H4 are outside this H0 record. H1 and H2 are documented in `h1_support_ablation.md` and `h2_irrelevant_covariate.md`. Remaining probes: `future_probes.md`.

## 8 Next

H0 is closed. Probes H1 and H2 are documented in `h1_support_ablation.md` and `h2_irrelevant_covariate.md`. Remaining deferred probes H3–H4 are recorded in `future_probes.md`. Still no Bucket 3 claim.